
Introduction to Database
CS350

Project

Deadline: Monday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details:

CRN: 50242

Name: Joud Abu Nameh

ID: s230010266

Name: Shahad Hazazy

ID: s220042807

Name: Jumanah Alfaeem

ID: s240030046

Name: Layan Alshehri

ID: s240017869

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
 - Database Design. (6 marks)
 - Database Implementation. (6 marks)
 - SQL Operations. (6 marks)
 - Project Reflection. (2 marks)
- Each leader in the group must submit one report about their chosen Project via the Blackboard (**Email submission will not be accepted and will be awarded ZERO marks**) containing the following:
 - A. Task 1 : database design
 - B. Task 2: database implementation
 - C. All requested queries/results
 - D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
 - E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You MUST show all your work, and text **must not** be converted into an image unless specified otherwise by the question.
- **Late submission will result in ZERO marks being awarded.**
- The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

*Learning**Outcome(s):**CLO1, CLO2, CLO3,**CLO4, CLO5*

Project Description

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Task 1 – Database Design (6 Marks)

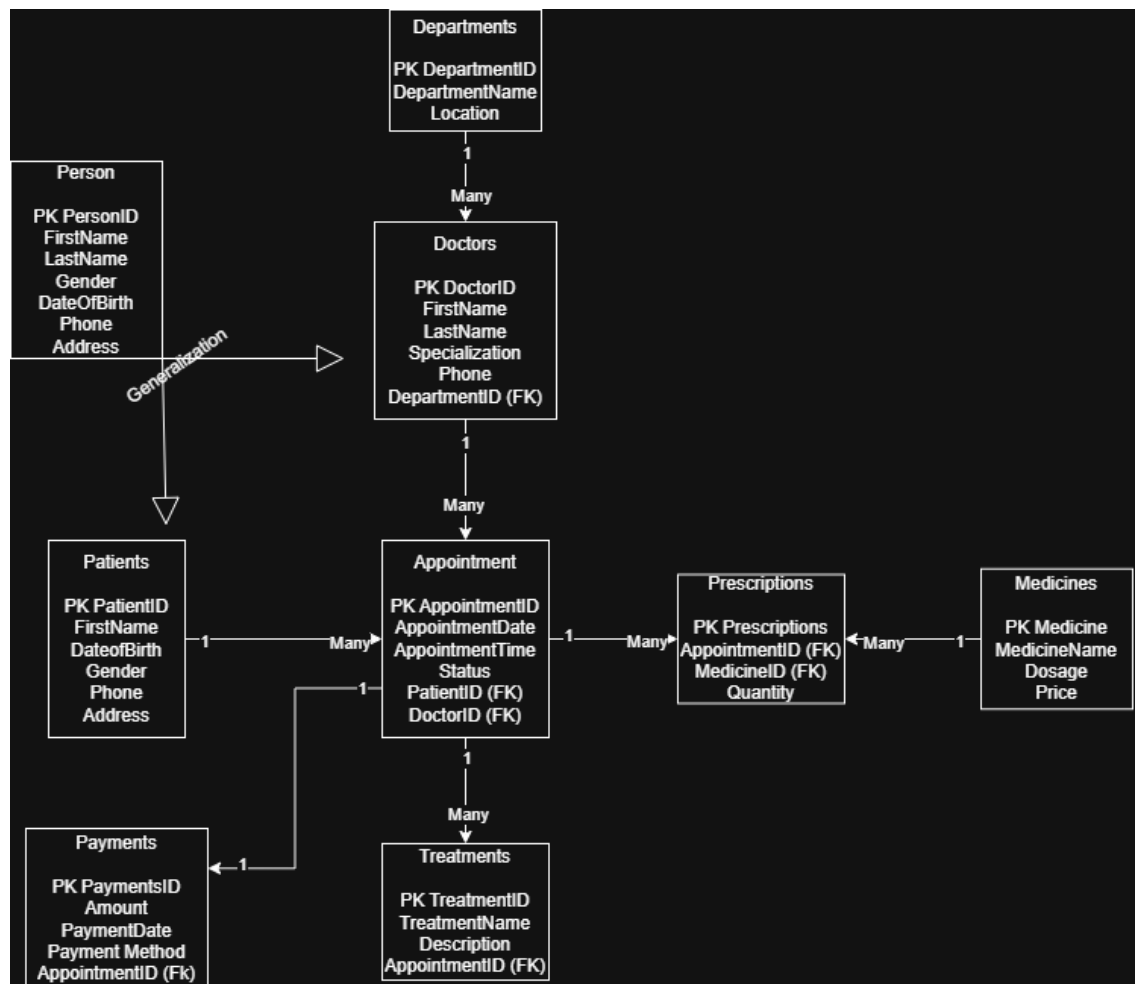
- Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.
- A complete ER/EER diagram.
- Primary and foreign keys.
- Appropriate relationships with cardinality.
- At least one specialization/generalization (EER feature).
- Any assumptions made during the design process.

The Smart Clinic Database System is designed for a private clinic that currently manages patient information manually. The proposed database organizes information related to patients, doctors, departments, appointments, treatments, medicines, prescriptions, and payments. The system is designed to improve data organization, reduce unnecessary data redundancy, and make information easier to store and retrieve. The database was designed using relational database principles. Primary keys are used to uniquely identify records, while foreign keys are used to connect related entities. The design was first developed as an ER/EER model and was then mapped into a relational schema before being implemented using MySQL.

Entity	Purpose
Patients	Stores information about clinic patients.
Doctors	Stores information about doctors and their specialties.
Departments	Stores the clinic's departments and their locations.
Appointments	

	Records appointments between patients and doctors.
Treatments	Stores information about medicines available in the clinic.
Medicines	Stores information about medicines available in the clinic.
Prescriptions	Connects patients and doctors with prescribed medicines.
Payments	Records payments made by patients.

Smart Clinic Database ER/EER Diagram



Departments(DepartmentID, DepartmentName, Location)

Doctors(DoctorID, FirstName, LastName, Specialty, DepartmentID)

Patients(PatientID, FirstName, LastName, Gender, DateOfBirth, Phone)

Appointments(AppointmentID, PatientID, DoctorID, AppointmentDate, AppointmentTime, Status)

Treatments(TreatmentID, PatientID, DoctorID, TreatmentDate, Description, Cost)

Medicines(MedicineID, MedicineName, Manufacturer, Price)

Prescriptions(PrescriptionID, PatientID, DoctorID, MedicineID, PrescriptionDate, Dosage)

Payments(PaymentID, PatientID, PaymentDate, Amount, PaymentMethod)

Each entity was assigned a primary key to uniquely identify its records. Foreign keys were used where entities needed to reference records in other tables.

Table	Primary Key	Foreign Keys
Departments	DepartmentID	-
Doctors	DoctorID	DepartmentID
Patients	PatientID	-
Appointments	AppointmentID	- PatientID, DoctorID
Medicines	MedicineID	-
Prescriptions	PrescriptionID	PatientID, DoctorID, MedicineID
Treatments	TreatmentID	PatientID, DoctorID
Payments	PaymentID	PatientID

The relationships between the entities were designed according to the clinic's workflow. A department can have multiple doctors, while each doctor is assigned to a department. Patients can have multiple appointments, and doctors can have multiple appointments. Patients and doctors can also be associated with multiple treatments. Prescriptions connect patients, doctors, and medicines, while payments are associated with patients.

Relationship	Cardinality
Department → Doctor	1:M
Patient → Appointment	1:M

Doctor → Appointment	1:M
Patient → Treatment	1:M
Doctor → Treatment	1:M
Patient → Prescription	1:M
Doctor → Prescription	1:M
Medicine → Prescription	1:M
Patient → Payment	1:M

A generalization/specialization feature was included in the EER model to satisfy the EER design requirement. The model represents common characteristics at the higher-level entity while allowing specialized entities to represent more specific roles. This structure demonstrates how entities with shared characteristics can be modeled without unnecessary duplication.

Design assumptions:

1. Each patient is identified by a unique PatientID.
2. Each doctor belongs to one department, while a department can contain multiple doctors.
3. A patient can have multiple appointments with doctors.
4. Each appointment is associated with one patient and one doctor.
5. A patient may receive multiple treatments during their clinic visits.
6. A prescription can associate a patient, doctor, and medicine.
7. A patient can make multiple payments.
8. Medicine information is stored separately from prescriptions to reduce unnecessary duplication.

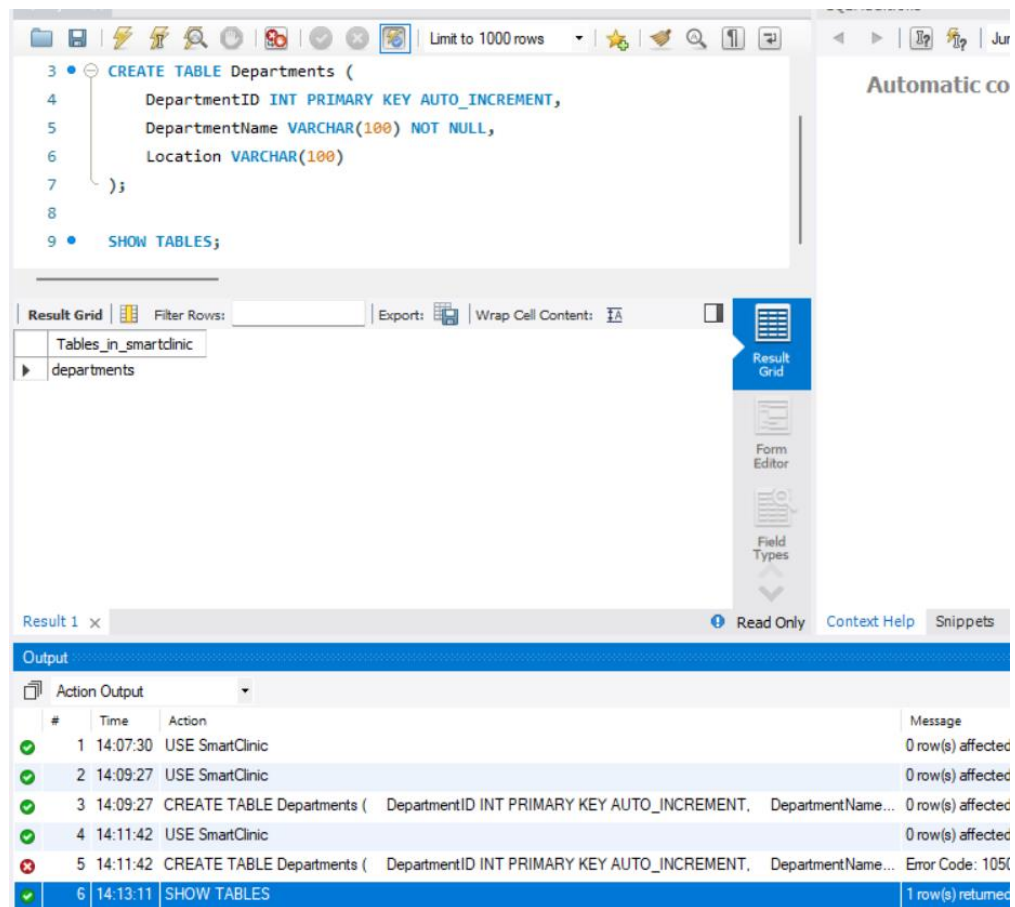
Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

- Create the database and all required tables.
- Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).
- Insert at least 5 records into each main table.

The Smart Clinic database was implemented using MySQL Workbench. The database was created using the name SmartClinic and was then selected for the creation of the required tables.

Creating the SmartClinic database in MySQL Workbench.



Eight tables were implemented to represent the main entities in the Smart Clinic system. Each table was created with appropriate attributes, data types, primary keys, and foreign keys where relationships were required.

Table	Main Key	Main Purpose
Departments	DepartmentsID	Stores clinic departments
Doctors	DoctorsID	Stores doctor information
Patients	PatientsID	Stores patient information
Appointments	AppointmentsID	

		Stores patient appointments
Medicines	MedicinesID	Stores medicine information
Prescriptions	PrescriptionsID	Stores prescriptions
Treatments	TreatmentsID	Stores treatments
Payments	PaymentID	Stores patient payments

Appropriate constraints were applied during implementation to maintain data integrity. Primary keys uniquely identify records, foreign keys maintain relationships between related tables, NOT NULL prevents required fields from being left empty, UNIQUE prevents duplicate values where appropriate, and AUTO_INCREMENT automatically generates unique identification numbers.

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Tables_in_smartclinic

- appointmentdetails
- appointments
- departments
- doctors
- medicines
- patients
- payments
- prescriptions
- treatments

Result Grid

Form Editor

Field Types

Result 9 x Read Only

Output

Action Output

#	Time	Action
✓ 8	11:14:06	SELECT COUNT(*) AS TotalAppointments FROM Appointments LIMIT 0, 1000
✓ 9	11:14:40	SELECT COUNT(*) AS TotalMedicines FROM Medicines LIMIT 0, 1000
✓ 10	11:15:04	SELECT COUNT(*) AS TotalPrescriptions FROM Prescriptions LIMIT 0, 1000
✓ 11	11:15:29	SELECT COUNT(*) AS TotalTreatments FROM Treatments LIMIT 0, 1000
✓ 12	11:16:05	SELECT COUNT(*) AS TotalPayments FROM Payments LIMIT 0, 1000

The screenshot displays a database management interface with two main sections. The top section shows a 'Result Grid' for the 'Payments' table, which contains 5 records. The bottom section shows an 'Action Output' log with 3 successful SQL queries.

Payments Table Data:

PaymentID	PatientID	Amount	PaymentDate	PaymentMethod
1	1	250.00	2026-08-01	Cash
2	2	500.00	2026-08-01	Credit Card
3	3	150.00	2026-08-02	Cash
4	4	300.00	2026-08-03	Debit Card
5	5	450.00	2026-08-04	Credit Card
* NULL	NULL	NULL	NULL	NULL

Action Output Log:

#	Time	Action
9	11:14:40	SELECT COUNT(*) AS TotalMedicines FROM Medicines LIMIT 0, 1000
10	11:15:04	SELECT COUNT(*) AS TotalPrescriptions FROM Prescriptions LIMIT 0, 1000
11	11:15:29	SELECT COUNT(*) AS TotalTreatments FROM Treatments LIMIT 0, 1000

The bottom section shows a 'Result Grid' for the 'Payments' table structure, which includes the following fields:

Field	Type	Null	Key	Default	Extra
PaymentID	int	NO	PRI	NULL	auto_increment
PatientID	int	YES	MUL	NULL	
Amount	decimal(10,2)	YES		NULL	
PaymentDate	date	YES		NULL	
PaymentMethod	varchar(50)	YES		NULL	

Each of the eight main tables was populated with at least five records using realistic sample data. The inserted data was reviewed using SELECT statements to verify that the records were successfully stored and that the relationships between related tables could be used during later SQL operations.

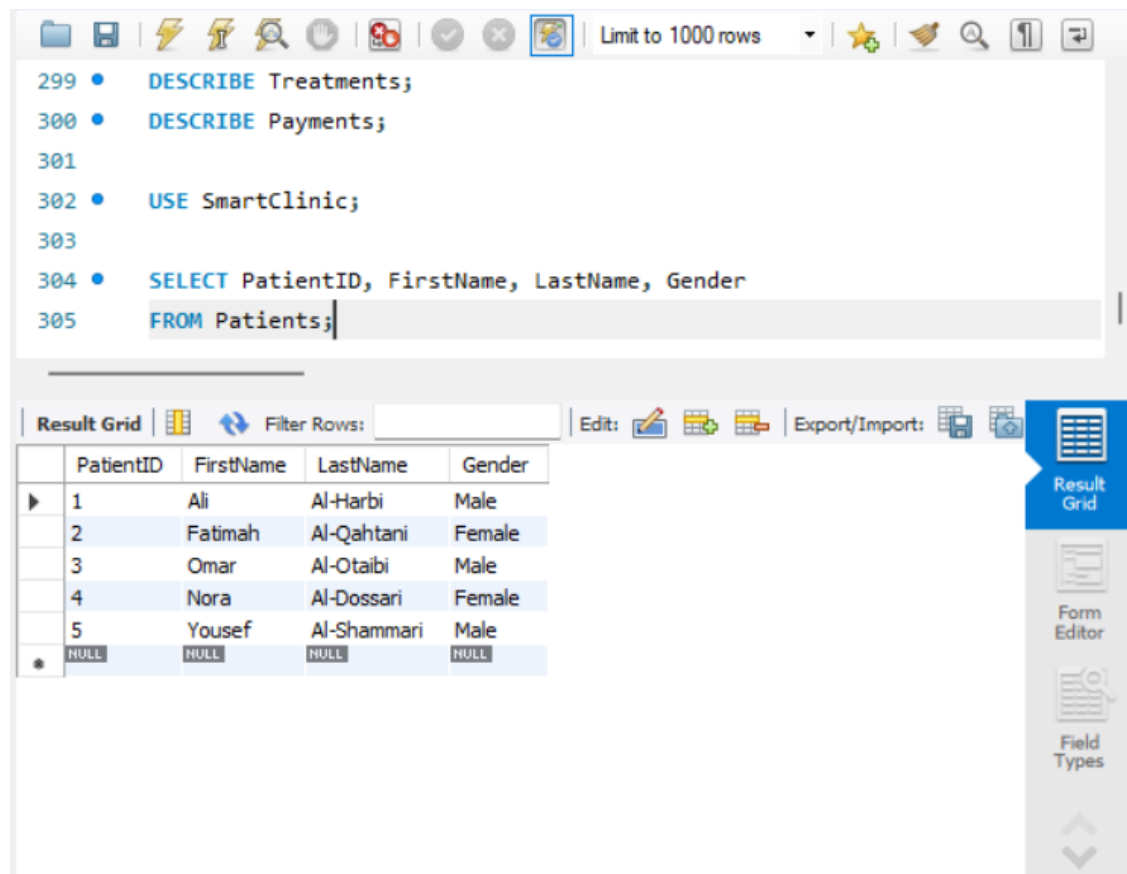
Appropriate data types and integrity constraints were used throughout the implementation. Primary keys were used to uniquely identify records, while foreign keys were used to maintain relationships between related tables. NOT NULL and UNIQUE constraints were applied where appropriate, and AUTO_INCREMENT was used for the identification fields.

The database contains eight main tables, and each table was populated with at least five records. The implemented primary keys, foreign keys, and other constraints were tested in MySQL Workbench to verify the integrity of the database structure.

Task 3 – SQL Operations (6 Marks)

Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

- Write SELECT statements.
- Write JOIN queries.
- Write nested queries.
- Use aggregate functions with GROUP BY.
- Write UPDATE and DELETE statements.
- Create one VIEW.
- Create one TRIGGER.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

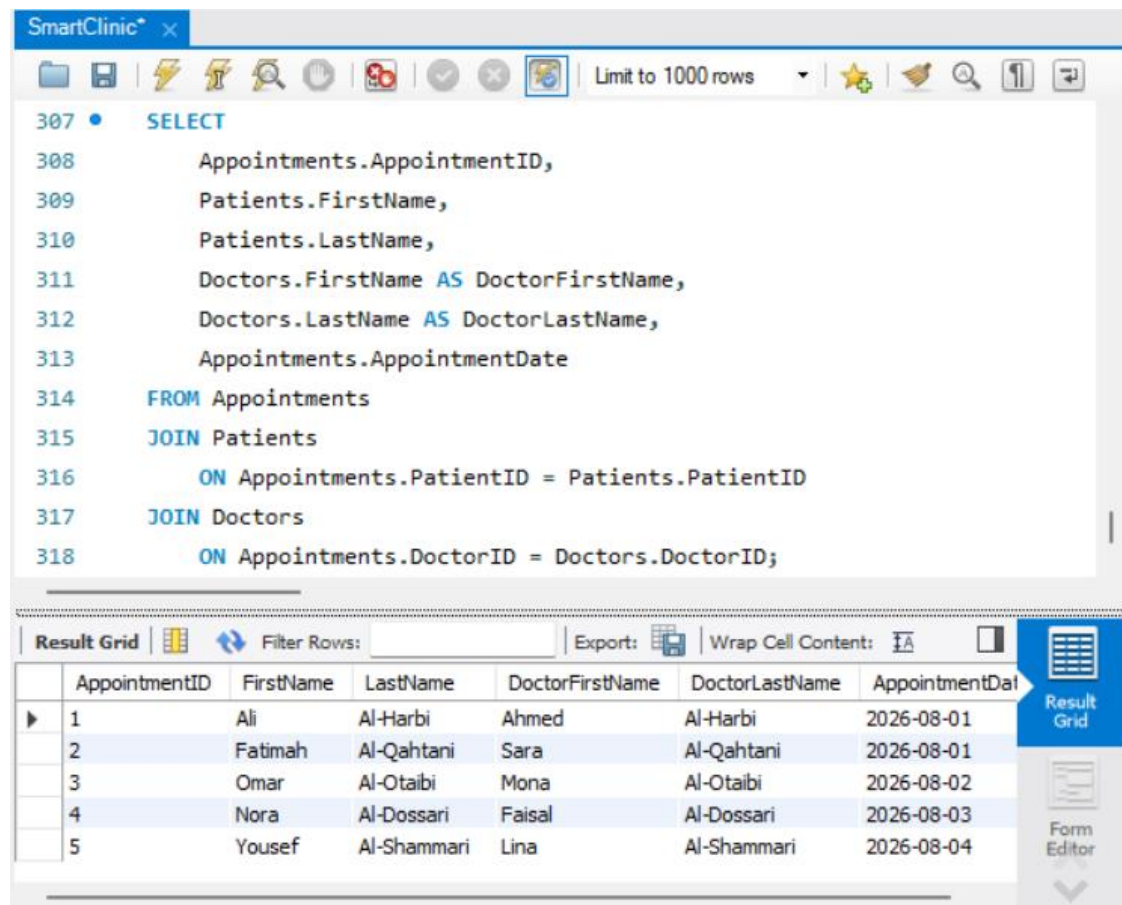
```
299 • DESCRIBE Treatments;  
300 • DESCRIBE Payments;  
301  
302 • USE SmartClinic;  
303  
304 • SELECT PatientID, FirstName, LastName, Gender  
305 FROM Patients;
```

Below the editor, the 'Result Grid' tab is active, displaying a table with 5 rows and 4 columns: PatientID, FirstName, LastName, and Gender. The first row is selected. The table data is as follows:

	PatientID	FirstName	LastName	Gender
▶	1	Ali	Al-Harbi	Male
	2	Fatimah	Al-Qahtani	Female
	3	Omar	Al-Otaibi	Male
	4	Nora	Al-Dossari	Female
	5	Yousef	Al-Shammari	Male
*	NULL	NULL	NULL	NULL

The right sidebar contains buttons for 'Result Grid', 'Form Editor', and 'Field Types'.

This query retrieves basic patient information from the Patients table. It displays the patient's ID, first name, last name, and gender.



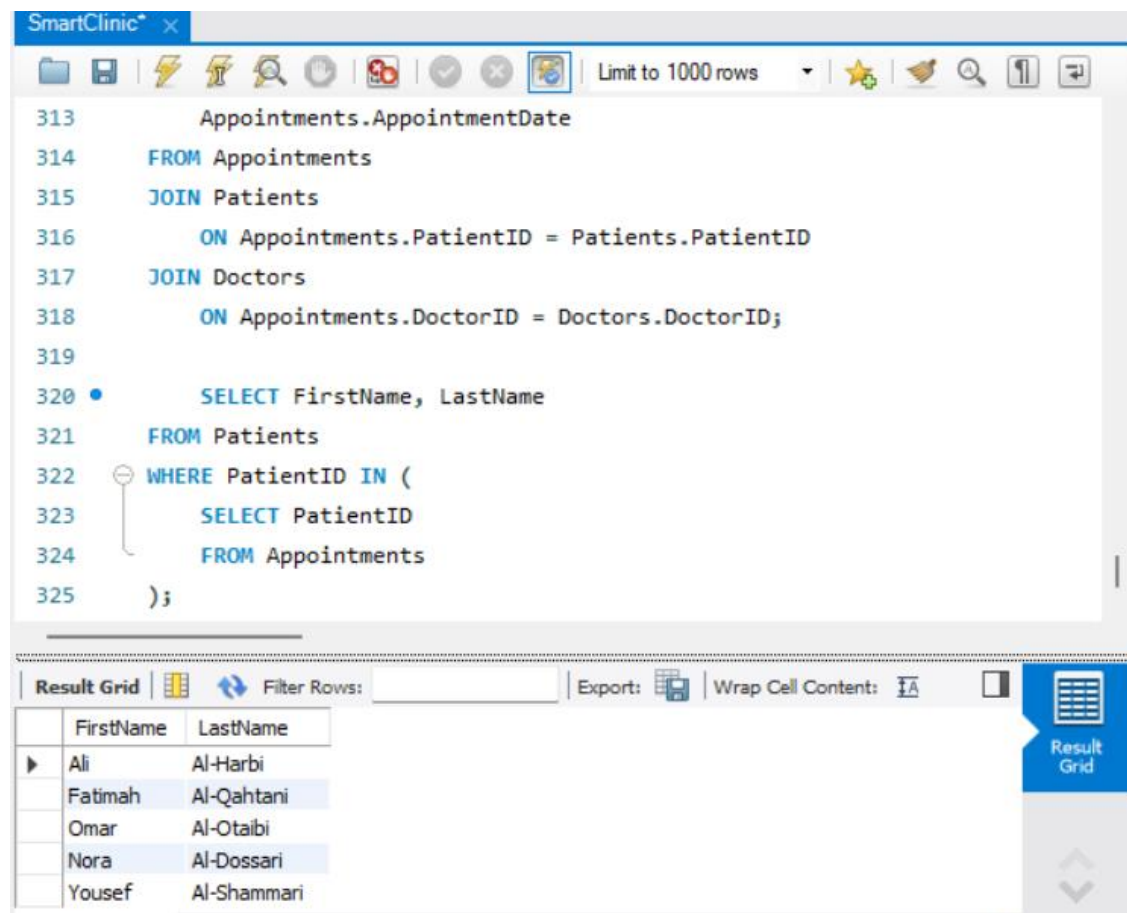
The screenshot shows the SmartClinic SQL editor interface. The query editor displays a SQL query that joins the Appointments, Patients, and Doctors tables. The query is as follows:

```
307 SELECT
308     Appointments.AppointmentID,
309     Patients.FirstName,
310     Patients.LastName,
311     Doctors.FirstName AS DoctorFirstName,
312     Doctors.LastName AS DoctorLastName,
313     Appointments.AppointmentDate
314 FROM Appointments
315 JOIN Patients
316     ON Appointments.PatientID = Patients.PatientID
317 JOIN Doctors
318     ON Appointments.DoctorID = Doctors.DoctorID;
```

Below the query editor, the Result Grid shows the output of the query. The grid has columns for AppointmentID, FirstName, LastName, DoctorFirstName, DoctorLastName, and AppointmentDate. The results are as follows:

	AppointmentID	FirstName	LastName	DoctorFirstName	DoctorLastName	AppointmentDate
▶	1	Ali	Al-Harbi	Ahmed	Al-Harbi	2026-08-01
	2	Fatimah	Al-Qahtani	Sara	Al-Qahtani	2026-08-01
	3	Omar	Al-Otaibi	Mona	Al-Otaibi	2026-08-02
	4	Nora	Al-Dossari	Faisal	Al-Dossari	2026-08-03
	5	Yousef	Al-Shammari	Lina	Al-Shammari	2026-08-04

• This query combines information from the Appointments, Patients, and Doctors tables. It shows which patient has an appointment with which doctor and the date of the appointment.



The screenshot shows the SmartClinic SQL editor interface. The SQL query is as follows:

```
313     Appointments.AppointmentDate
314 FROM Appointments
315 JOIN Patients
316     ON Appointments.PatientID = Patients.PatientID
317 JOIN Doctors
318     ON Appointments.DoctorID = Doctors.DoctorID;
319
320 •   SELECT FirstName, LastName
321 FROM Patients
322 WHERE PatientID IN (
323     SELECT PatientID
324     FROM Appointments
325 );
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has two columns: 'FirstName' and 'LastName'. The results are as follows:

FirstName	LastName
Ali	Al-Harbi
Fatimah	Al-Qahtani
Omar	Al-Otaibi
Nora	Al-Dossari
Yousef	Al-Shammari

This nested query identifies patients who have at least one appointment. The inner query retrieves PatientIDs from the Appointments table, and the outer query retrieves the corresponding patient names.

```
326
327 • SELECT DoctorID, COUNT(*) AS TotalAppointments
328 FROM Appointments
329 GROUP BY DoctorID;
```

Result Grid

DoctorID	TotalAppointments
1	1
2	1
3	1
4	1
5	1

Result 15 x Read Only

This query counts the number of appointments associated with each doctor. The GROUP BY clause groups the appointments according to DoctorID.

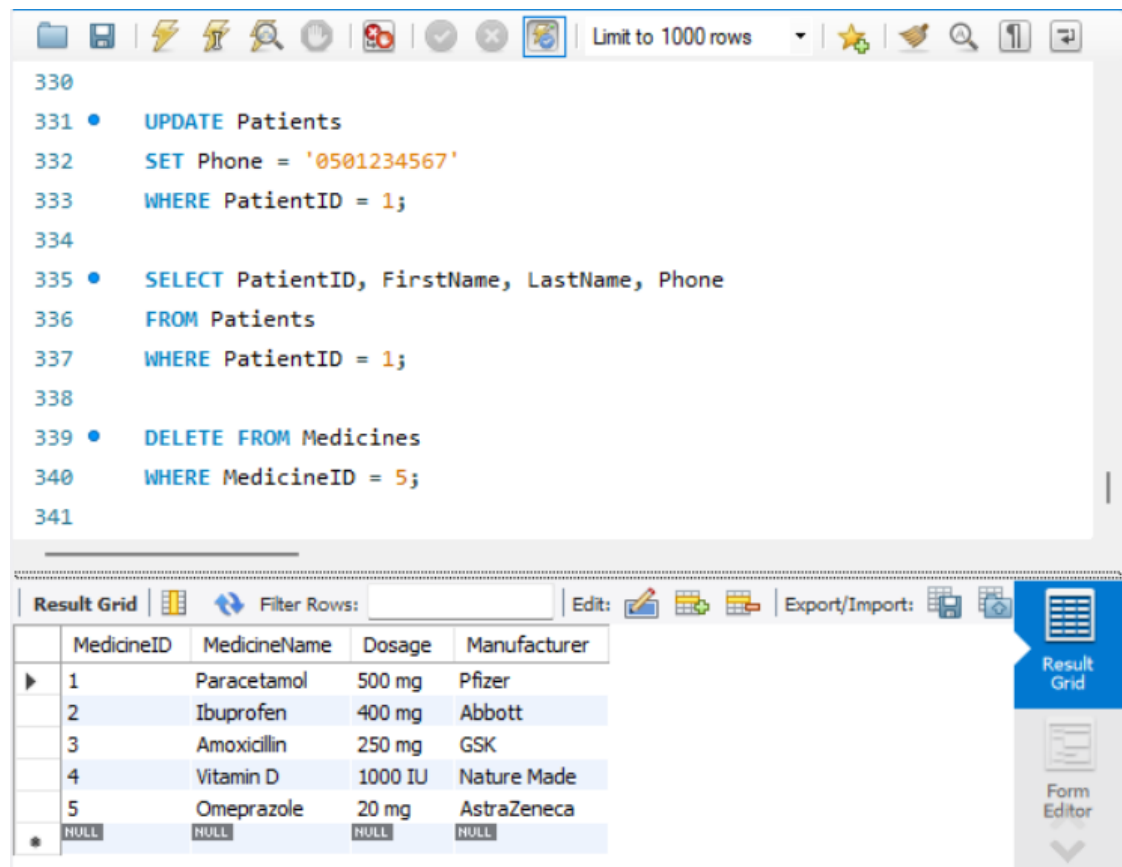
```
327 • SELECT DoctorID, COUNT(*) AS TotalAppointments
328 FROM Appointments
329 GROUP BY DoctorID;
330
331 • UPDATE Patients
332 SET Phone = '0501234567'
333 WHERE PatientID = 1;
334
335 • SELECT PatientID, FirstName, LastName, Phone
336 FROM Patients
337 WHERE PatientID = 1;
```

Result Grid

PatientID	FirstName	LastName	Phone
1	Ali	Al-Harbi	0559999999
NULL	NULL	NULL	NULL

Patients 16 x Apply

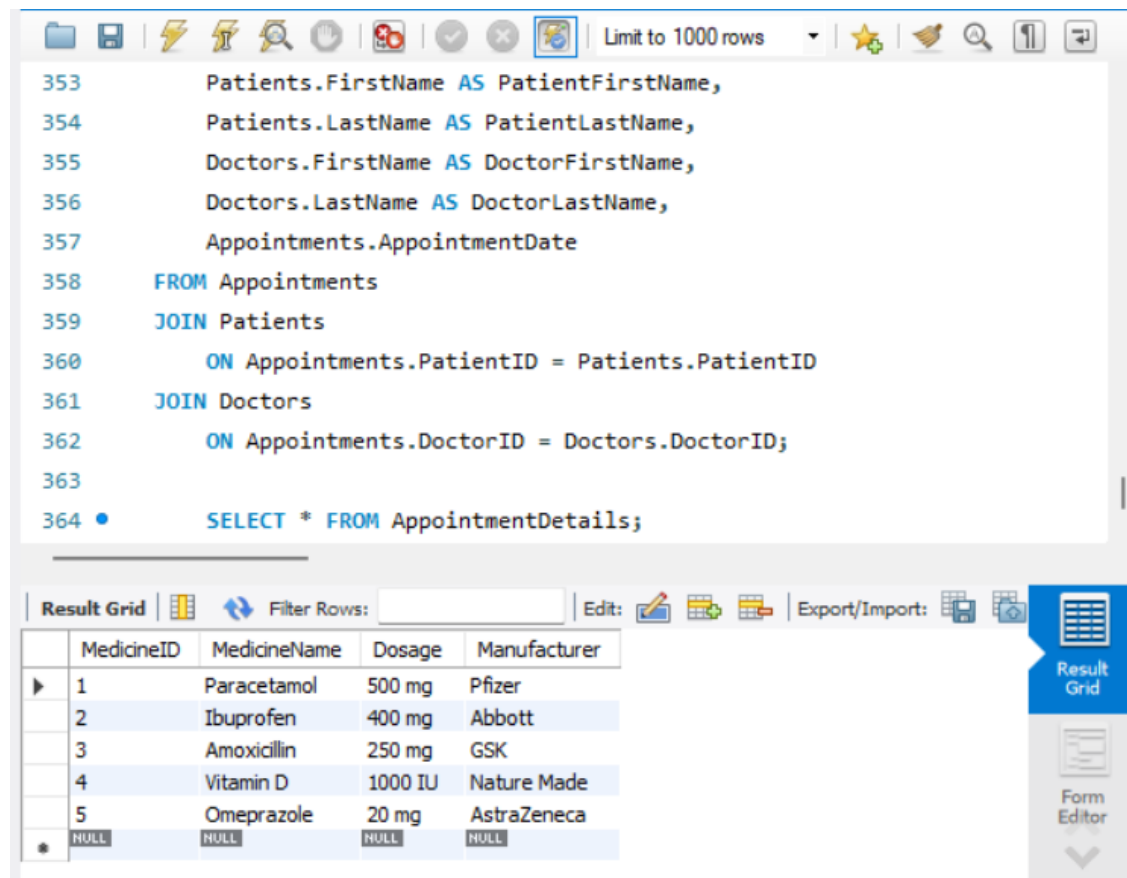
This query updates the phone number of a specific patient. The SELECT statement is then used to verify that the change was successfully applied.



```
330
331 • UPDATE Patients
332   SET Phone = '0501234567'
333   WHERE PatientID = 1;
334
335 • SELECT PatientID, FirstName, LastName, Phone
336   FROM Patients
337   WHERE PatientID = 1;
338
339 • DELETE FROM Medicines
340   WHERE MedicineID = 5;
341
```

	MedicineID	MedicineName	Dosage	Manufacturer
▶	1	Paracetamol	500 mg	Pfizer
	2	Ibuprofen	400 mg	Abbott
	3	Amoxicillin	250 mg	GSK
	4	Vitamin D	1000 IU	Nature Made
	5	Omeprazole	20 mg	AstraZeneca
*	NULL	NULL	NULL	NULL

This query demonstrates how a specific medicine record can be deleted from the database. The SELECT statement verifies that the selected record has been removed.



The screenshot displays a SQL IDE interface. The top pane shows a SQL query (lines 353-364) that joins the Patients, Doctors, and Appointments tables to retrieve appointment details. The bottom pane shows the 'Result Grid' with 5 rows of data. The columns are MedicineID, MedicineName, Dosage, and Manufacturer. The data rows are: 1 Paracetamol 500 mg Pfizer, 2 Ibuprofen 400 mg Abbott, 3 Amoxicillin 250 mg GSK, 4 Vitamin D 1000 IU Nature Made, and 5 Omeprazole 20 mg AstraZeneca. A final row shows NULL values for all columns.

```
353 Patients.FirstName AS PatientFirstName,  
354 Patients.LastName AS PatientLastName,  
355 Doctors.FirstName AS DoctorFirstName,  
356 Doctors.LastName AS DoctorLastName,  
357 Appointments.AppointmentDate  
358 FROM Appointments  
359 JOIN Patients  
360     ON Appointments.PatientID = Patients.PatientID  
361 JOIN Doctors  
362     ON Appointments.DoctorID = Doctors.DoctorID;  
363  
364 • SELECT * FROM AppointmentDetails;
```

	MedicineID	MedicineName	Dosage	Manufacturer
▶	1	Paracetamol	500 mg	Pfizer
	2	Ibuprofen	400 mg	Abbott
	3	Amoxicillin	250 mg	GSK
	4	Vitamin D	1000 IU	Nature Made
	5	Omeprazole	20 mg	AstraZeneca
*	NULL	NULL	NULL	NULL

This view combines appointment information with the related patient and doctor information. It provides a convenient way to retrieve appointment details without repeatedly writing the same JOIN statements.

The screenshot displays the SQL Server Enterprise Manager interface. The top toolbar includes icons for file operations, execution, and search. The main window shows a SQL script with the following code:

```
370 FOR EACH ROW
371 BEGIN
372     IF NEW.Amount <= 0 THEN
373         SIGNAL SQLSTATE '45000'
374         SET MESSAGE_TEXT = 'Payment amount must be greater than zero';
375     END IF;
376 END//
377
378 DELIMITER ;
379
380 • SHOW TRIGGERS;
381
```

Below the script, the 'Result Grid' tab is active, showing a table with the following data:

Trigger	Event	Table	Statement
CheckPaymentAmount	INSERT	payments	BEGIN IF NEW.Amount < 0 THEN SIGNAL SQLS... BE

This trigger prevents invalid payment amounts from being inserted into the Payments table. It checks the payment amount before insertion and generates an error if the amount is zero or negative.

Verification:

The screenshot displays the SQL Server Enterprise Manager interface. The main window shows a SQL query:

```
382 • SELECT COUNT(*) AS TotalMedicines
383 FROM Medicines;
```

Below the query, the 'Result Grid' tab is active, showing a table with the following data:

TotalMedicines
5

Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

1. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
 2. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
 3. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
- What would you do differently if you were to redo this project with more time or resources?

Working on the Smart Clinic Database System gave our team a better understanding of how database concepts are applied in a complete project. One of the main challenges we faced was deciding how the different entities should be connected, especially appointments, treatments, prescriptions, medicines, and payments. At first, some possible relationships could have created unnecessary duplication or made the database more complicated. We resolved this by reviewing the clinic's workflow, discussing the purpose of each entity, and checking the relationships against normalization principles. We then revised the ER/EER diagram before completing the MySQL implementation.

We chose to complete the database design before implementing all of the tables in MySQL. Another option would have been to create the tables immediately and change them whenever a problem was discovered. Although that approach might have been faster at the beginning, it could have caused more changes and inconsistencies later. Finalizing the ER/EER diagram, keys, relationships, and relational structure first took more planning, but it gave us a clearer structure to follow during implementation.

During the project, we used Chat2DB as an AI assistant for specific tasks such as understanding SQL syntax, troubleshooting errors, clarifying database concepts, and

improving some of the wording in the documentation. We also used AI assistance while preparing and reviewing some SQL operations. We did not rely on the generated responses without checking them. SQL statements were tested in MySQL Workbench, and the results were compared with our database design and project requirements. We modified suggestions whenever necessary to make sure they matched our Smart Clinic database.

If we had more time and resources, we would improve the project by adding more realistic sample data and performing more extensive testing with different situations. We would also spend more time refining the database design and documentation before the final submission. Overall, completing the project helped us understand the connection between ER/EER modeling, relational database design, SQL implementation, and database testing.

Deliverables

1. **Project Report (PDF)** – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.
2. **SQL Script (.sql)** – Complete SQL implementation of the project.
3. **GitHub Repository** – Repository link with the complete project and commit history.
4. **Mid-Project Progress Report** – Summary of progress, challenges, decisions, and team contributions.
5. **Project Artifacts**
 - Live report document (with edit history): [link]
 - GitHub/GitLab repository (with commit history): [link]
 - Google Colab notebook (if applicable): [link]

Our mid report was written in Google Docs.

Github Link: <https://github.com/collegejoud/Smart-Clinic-Database-System>

GoogleDocs Document (mid report part only): <https://docs.google.com/document/d/1-KvSMAModsbiTNgIgf6G0Ne-RzVRgdeuzVCXc06--10/edit?usp=drivesdk>